

Correction — Baccalauréat NSI 2026

Sujet 26-NSIJ2JA1

Exercice 1 — Le Taquin (6 points)

Question 1

`tab = [5, 3, 8, 0, 1, 2, 7, 6, 4]`

La case numéro 2 est à l'indice 5 (`tab[5] = 2`).

Après exécution de `tab[3] = tab[4]` puis `tab[4] = 0` :

```
tab = [5, 3, 8, 1, 0, 2, 7, 6, 4]
```

Grille obtenue :

5	3	8
1		2
7	6	4

Question 3 — Expression booléenne « position gagnante »

```
tab == [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Question 4 — Méthode `est_gagnant`

```
def est_gagnant(self):  
    return self.tab == [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Question 5 — Compléter indice (lignes 3 et 5)

```
def indice(self, numero):  
    assert type(numero) == int, 'numero doit être entier'  
    assert 0 <= numero <= 8, 'numero de case non valide' # ligne 3  
    i = 0  
    while self.tab[i] != numero: # ligne 5  
        i = i + 1  
    return i
```

Question 6 — Compléter jouer (lignes 2 à 6)

```
def jouer(self, numero):  
    if self.est_possible(numero): # ligne 2  
        i = self.indice(numero)  
        j = self.indice(0)  
        self.tab[j] = numero # ligne 5  
        self.tab[i] = 0 # ligne 6
```

Question 7 — Compléter melanger (lignes 4, 5, 6, 9)

```
def melanger(self, n):  
    precedent = None  
    i = 0  
    while i < n: # ligne 4
```

```
possibilites = self.coups_possibles() # ligne 5
choix = random.choice(possibilites) # ligne 6
if choix != precedent:
    self.jouer(choix)
    precedent = choix # ligne 9
    i = i + 1
```

■ L'incrément de `i` doit être à l'intérieur du `if`, afin que le compteur n'avance que lorsqu'un coup est effectivement joué.

Question 8 — Résolution automatique : ordre des coups

Pile après mélange + coup joueur (bottom → top) : 1, 4, 5, 2.

La résolution dépile en ordre LIFO :

Étape	Coup joué	État de la pile (top → bottom)
Départ	—	[2, 5, 4, 1] (top = 2)
1	2	[5, 4, 1]
2	5	[4, 1]
3	4	[1]
4	1	[] → pile vide, position gagnante ✓

Question 9 — Méthode `resoudre`

```
def resoudre(self):
    self.mode_resolution = True
    while not self.est_gagnant():
        coup = self.pile.depiler()
        print(coup)
        self.jouer(coup)
```

Question 10 — Pourquoi l'oubli de `mode_resolution` cause une non-termination

Sans passer en mode résolution, chaque appel à `jouer()` réempile le coup dans la pile. On dépile un coup, on le rejoue, il se réempile aussitôt : la pile ne se vide jamais et la boucle `while not self.est_gagnant()` tourne indéfiniment.

Question 11 — Optimisation de `jouer` à partir de la ligne 8

```
if not self.mode_resolution:
    if not self.pile.est_vide():
        sommet = self.pile.depiler()
        if sommet != numero:
            self.pile.empiler(sommet)
            self.pile.empiler(numero)
        # sinon les deux coups s'annulent, on n'empile rien
    else:
        self.pile.empiler(numero)
```

Exercice 2 — Bases de données et graphes (6 points)

Partie A — SQL

Question 1

id_pers est une clé étrangère dans la table **participation**.

Question 2

id_partie ne peut pas être clé primaire de participation car plusieurs joueurs participent à la même partie : id_partie n'est donc pas unique dans cette table. La clé primaire est le couple (**id_partie, id_pers**).

Question 3

```
INSERT INTO personne (id_pers, pseudo_pers, date_pers)
VALUES (42, 'theorie', '2022-12-14');
```

Question 4

```
SELECT participation.id_partie
FROM participation
JOIN personne ON participation.id_pers = personne.id_pers
WHERE personne.pseudo_pers = 'test';
```

Question 5

Il faut d'abord supprimer les participations pour respecter l'intégrité référentielle :

```
DELETE FROM participation
WHERE id_pers = 8;
DELETE FROM personne
WHERE id_pers = 8;
```

Partie B — Graphe et ordre alphabétique

Question 6 — Compléter indice (lignes 3 et 4)

```
def indice(lettre, ordre):
    for i in range(len(ordre)):
        if ordre[i] == lettre: # ligne 3
            return i # ligne 4
```

Question 7 — Compléter comparer (lignes 7, 9, 11)

```
# ligne 7 (i1 < i2 : mot1 avant mot2)
return True
# ligne 9 (i1 > i2 : mot1 après mot2)
return False
# ligne 11 (boucle terminée sans différence : mot1 est préfixe de mot2)
return len(mot1) <= len(mot2)
```

Question 8 — premiere_diff

```
def premiere_diff(mot1, mot2):
    i = 0
    while i < len(mot1) and i < len(mot2) and mot1[i] == mot2[i]:
        i += 1
    return i
```

Vérification : premiere_diff("oou", "ooai") = 2 ✓ | premiere_diff("aio", "aioiee") = 3 ✓

Question 9 — dico_adj(mots_exemple)

```
{
    'u': ['a', 'y'],
    'y': ['i', 'a'],
    'a': ['i'],
```

```
'o': ['u'],
'e': ['a']
}
```

Question 10 — Type de parcours

C'est un **parcours en profondeur (DFS)**. La fonction s'appelle récursivement sur chaque voisin avant d'ajouter le sommet courant à tri, ce qui est caractéristique d'un tri topologique par DFS.

Question 11 — Fonction trier

```
def trier(mots):
    adj = dico_adj(mots)
    tri = []
    deja_vus = []
    voyelles = ['a', 'e', 'i', 'o', 'u', 'y']
    for v in voyelles:
        if v not in deja_vus:
            deja_vus.append(v)
            parcours(adj, v, deja_vus, tri)
    tri.reverse()
    return tri
```

Exercice 3 — Robots et routage RIP (8 points)

Partie A — Gestion des déplacements

Question 1 — Parcours pour '4(AG)'

'4(AG)' = répéter 4 fois le bloc AG = AGAGAGAG.

Le robot trace un **carré** dans le sens antihoraire et revient à sa position initiale.

Question 2 — caracteres_valides, ligne 4

```
return intrus == []
```

Question 3 — entiers_valides, lignes 4, 8, 9

```
def entiers_valides(chaine):
    chiffres = '0123456789'
    if chaine[len(chaine)-1] in chiffres: # fin de chaîne = chiffre
        return False # ligne 4
    for indice in range(1, len(chaine)):
        if chaine[indice] == ')':
            if chaine[indice - 1] in chiffres:
                return False # lignes 8-9
    return True
```

Question 4 — parenthesage_correct

```
def parenthesage_correct(chaine):
    parenthese = 0
    for c in chaine:
        if c == '(':
            parenthese += 1
        elif c == ')':
            parenthese -= 1
```

```

    if parenthese < 0:
        return False
    return parenthese == 0

```

Question 5 — lire_nombre('AD179AGA', 2) : itérations

Itération	nombre	indice
1	'1'	3
2	'17'	4
3	'179'	5

Retourne (179, 4) ✓

Question 6 — lire_bloc, lignes 7, 8, 9

```

bloc = bloc + caractere # ligne 7
indice = indice + 1 # ligne 8
caractere = chaine[indice] # ligne 9

```

Question 7 — lire_parcours, lignes 12, 16, 20

```

t = lire_nombre(chaine, indice) # ligne 12 (cas répétition)
t = lire_bloc(chaine, indice) # ligne 16 (cas début de bloc)
lire_parcours(bloc) # ligne 20 (exécuter le bloc nombre fois)

```

Partie B — Routage RIP

Question 8 — Modifications après ajout du lien 4 ↔ 46

Le robot 4 est désormais voisin direct de 46. Pour le robot 91, atteindre 46 via 4 ne prend plus que **2 sauts** (au lieu de 3).

Modification : destination 46 → prochain = 4, distance = 2.

Question 9 — Modifications après arrivée du robot 87

Le robot 87 contacte 91 et lui communique sa table :

destination	prochain robot	distance
87	87	1
63	87	2
36	87	3

Partie C — Programmation du routage

Question 10 — ajouter_voisin

```

def ajouter_voisin(self, id_voisin):
    self.table_routage[id_voisin] = {"prochain": id_voisin, "distance": 1}

```

Question 11 — nombre_sauts, ligne 5

```

return self.table_routage[identifiant]["distance"]

```

Question 12 — voisins

```

def voisins(self):
    return [dest for dest in self.table_routage
            if self.table_routage[dest]["prochain"] == dest]

```

```
and self.table_routage[dest]["distance"] == 1]
```

Question 13 — communiquer_extrait_table

```
def communiquer_extrait_table(self, voisin):  
    extrait = {}  
    for dest, info in self.table_routage.items():  
        if info["prochain"] != voisin:  
            extrait[dest] = info  
    return extrait
```